SIEM Deployment & Log Aggregation

SOC Lab Report

Prepared By: Turki Alotaibi

Environment: Two Ubuntu servers & windows 11 workstation & Splunk Enterprise & Universal Forwarders

1. Executive Summary

The objective of this laboratory deployment was to design, construct, and validate a centralized Security Information and Event Management (SIEM) pipeline capable of aggregating event logs from diverse operating systems. In modern enterprise architecture, visibility is the baseline of proactive defense. Without synchronized log collection, identifying sophisticated lateral movement or multi-stage cyber threats becomes virtually impossible.

To simulate this enterprise visibility, a dedicated central logging framework was established using **Splunk Enterprise** hosted on an Ubuntu Server instance. To supply this indexer with telemetry, end-tier infrastructure assets—consisting of a Windows 11 workstation and an autonomous Ubuntu server—were provisioned with

lightweight **Splunk Universal Forwarders**. Custom target indices were structurally segregated on the central SIEM to optimize data retrieval, maximize query parsing speeds, and enforce data isolation boundaries.

2. Architecture Infrastructure Design

The environment is organized into a classical hub-and-spoke model, decoupling telemetry collection from data analysis. By embedding lightweight forwarding agents directly into the host OS, log processing overhead is minimalized on the target endpoints while shifting raw indexing workloads onto the centralized core.

Node Asset Name	Operating System	Role in Architecture	Telemetry Source / Configuration
Splunk-Indexer-SRV	Ubuntu Server LTS	Central Indexer / SIEM Receiver	Port `9997` (Listening Receiver Enabled)
Win11-Endpoint	Windows 11 Professional	Data Producer / Client Node	Security, Application, and System Event Logs
Linux-Endpoint-SRV	Ubuntu Server	Data Producer / Client Node	Syslog, Auth.log, and Audit Telemetry

3. Centralized SIEM Core Implementation

The primary objective commenced with provisioning the central management plane. An Ubuntu Server distribution was fortified and configured to support the heavy I/O capabilities required by Splunk Enterprise.

3.1 Splunk Enterprise Deployment & Initialization

Following the extraction of the Splunk Debian package binaries, initialization binaries were systematically run to enable system startup routines. The administrative web portal was tethered to local interfaces, securing high-level analytical capabilities via the graphic web panel.

3.2 Enabling Network Listening Protocols

Before clients could stream raw structural events, the Splunk instance required an operational listener loop to bind incoming TCP traffic. Using the Splunk settings engine, a network receiving port was specified at standard TCP port `9997`. This bound a persistent listening engine to process streaming raw logs from inbound nodes.

> **Security Architecture Policy:** Network ingress rules and hosting firewalls (UFW) were configured explicitly to permit unidirectional traffic flowing from the endpoint network boundaries toward the SIEM core over TCP port 9997.

4. Storage Layer Architecture & Data Segregation

Storing different telemetry profiles inside a single logical index degrades search response times and dilutes index-level retention policies. To counter this, logical storage spaces were structurally isolated by creating two unique indices dedicated to targeted operating system architectures:

- `windows` **Index:** Designated exclusively for structured Event Log channels originating from the Windows 11 platform (Security, System, Application alerts).
- `linux` **Index:** Formatted specifically for parsing unstructured system events, shell audit records, and security authentication frameworks (`auth.log` , `syslog`) driven by the Ubuntu client node.

This structural segregation ensures that Security Processing Language (SPL) queries run optimally, yielding sub-second query execution times during incident analysis exercises.

5. Endpoint Agent Deployment & Forwarding Pipelines

With the central ingest layer active and listening, the focus shifted to endpoint collection agents. This was accomplished by deploying the **Splunk Universal Forwarder** across the assets to push log queues out to the central receiver.

5.1 Windows 11 Workspace Configuration

The installation process on the Windows 11 client required mapping specific hooks into the underlying Windows Event Log structure.

1. The Universal Forwarder MSI package was executed with localized administrative permissions.
2. The engine directed outgoing traffic streams directly to the IP address of the central Ubuntu Splunk server via destination port `9997`.
3. The host-level `inputs.conf` manifest file was tailored to hook explicitly into foundational Windows security primitives, streaming live channels directly into the dedicated `windows` index.

5.2 Linux Endpoint Server Configuration

For the standalone Ubuntu client asset, the installation leaned heavily on non-graphical configuration management commands.

1. The lightweight forwarder application tarball was deployed and extracted within the endpoint's localized `/opt` storage pathways.
2. The active agent was coupled to the logging core by setting the deployment target pointer via command line interface:

   ```
   ./splunk add forward-server <Splunk_Server_IP>:9997
   ```

3. Local tracking hooks were systematically appended to log configurations via the local inputs file, pointing directly to critical system paths:

   ```
   /var/log/syslog
   /var/log/auth.log
   ```

4. All downstream streams originating from this node were assigned metadata targeting the `linux` index.

6. Operational Validation & Pipeline Testing

To confirm the structural integrity of the logging pipeline, validation steps were performed via the main Search & Reporting web application. Analytical lookups were executed to verify active log streams and ensure zero data loss at the ingest interface.

6.1 Linux Stream Ingestion Verification

An operational test search query was evaluated against the Linux index storage boundaries:

```
index="linux" | stats count by host, sourcetype
```

The search successfully populated log events mapping back to the target Ubuntu endpoint, confirming that local authentication events and standard system mutations are accurately streaming into the index.

6.2 Windows Stream Ingestion Verification

Similarly, checking the integrity of the Windows client data channel was validated by isolating the Windows index:

```
index="windows" | top limit=10 EventCode
```

The results displayed authentic Windows security system matrices, registering tracking codes such as `4624` (Successful Account Logon) and `4625` (Failed Account Logon). This indicates complete log fidelity.

7. Analytical Conclusion

The implementation of this laboratory infrastructure marks the successful execution of an elastic, enterprise-grade log pipeline. By decoupling data gathering via decentralized Universal Forwarders and centralizing analytical metrics into structured, isolated indices within Splunk Enterprise, the lab establishes a foundational detection framework. This ecosystem is primed for advanced security monitoring, behavioral analysis modeling, and simulated Blue-Team incident response exercises.